

E1 — Propuesta de Solución

Grupo 5 — Capa de Transporte: análisis de TCP y UDP con detección de escaneo de puertos

Integrantes	[Rick Rodriguez] · [Integrante 2] · [Integrante 3]
Repositorio	[URL del repositorio Git público — p. ej. https://github.com/usuario/g5-transporte]
Fecha	12 de junio de 2026

1. Comprensión del problema y del protocolo

Problema. La capa de transporte de la pila TCP/IP es responsable de la comunicación extremo a extremo entre procesos. Nuestro objetivo es construir un sistema propio que evidencie, sobre tráfico real y operando directamente sobre los campos del protocolo, el comportamiento de TCP y UDP, y que detecte una anomalía característica de esta capa: el **escaneo de puertos**.

TCP (RFC 9293, que consolida y reemplaza al RFC 793). Protocolo orientado a conexión, fiable, full-duplex y con control de flujo. Su cabecera mínima de 20 bytes incluye puertos de origen y destino, número de secuencia, número de acuse (ACK), desplazamiento de datos, los bits de control (URG, ACK, PSH, RST, SYN, FIN), el tamaño de ventana, la suma de verificación, el puntero urgente y las opciones. La conexión se establece mediante el **acuerdo de tres vías** (SYN → SYN-ACK → ACK), se gobierna por una máquina de estados (LISTEN, SYN-SENT, SYN-RECEIVED, ESTABLISHED, FIN-WAIT, TIME-WAIT, etc.), aplica control de flujo por ventana deslizante y control de congestión (RFC 5681), y se cierra de forma ordenada con FIN o abrupta con RST.

UDP (RFC 768). Protocolo sin conexión y sin garantías de entrega, orden ni control de flujo. Su cabecera es de solo 8 bytes (puertos de origen y destino, longitud y suma de verificación). Su baja sobrecarga es la base de la comparación con el modelo fiable de TCP.

Anomalía a estudiar — escaneo de puertos. El escaneo explota la semántica de respuestas de la capa: en un escaneo SYN (“half-open”) un puerto abierto responde SYN-ACK y uno cerrado responde RST, sin completar nunca el acuerdo; los escaneos FIN/NULL/Xmas se basan en que los puertos cerrados responden RST y los abiertos descartan el segmento; el escaneo UDP infiere el estado a partir de la ausencia de respuesta o de un mensaje ICMP “port unreachable”. Como extensión, contemplamos analizar la inundación SYN (RFC 4987) por su relación directa con la cola de conexiones semiabiertas.

2. Arquitectura propuesta

El sistema se organiza en una tubería de seis componentes que cubren los tres requisitos mínimos del enunciado. La generación de tráfico y la captura producen los datos; el **parser de bajo nivel** y el reconstructor de estados constituyen la implementación propia (no delegada en librerías de alto nivel); el detector de anomalías y la validación contra Wireshark cierran el análisis.

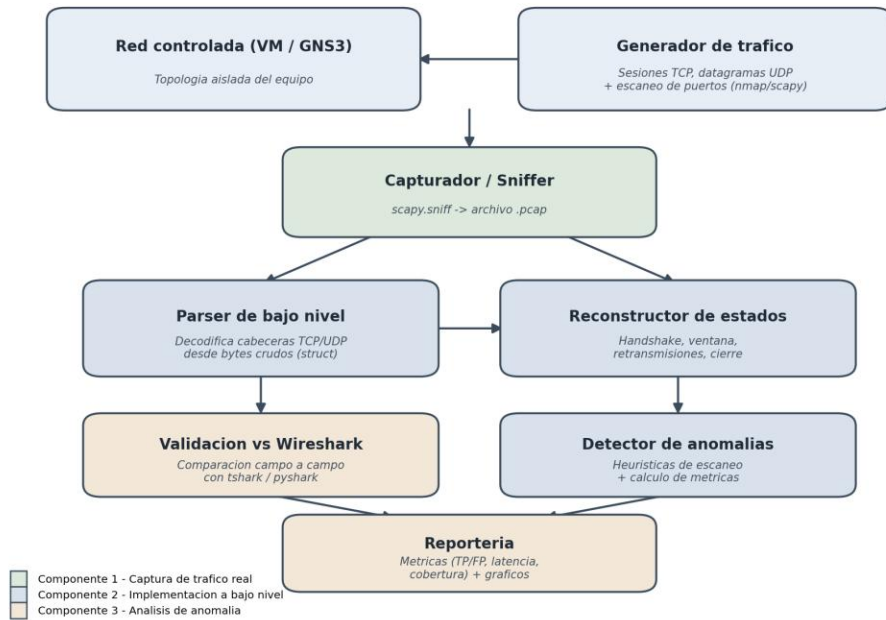


Figura 1. Diagrama de componentes del sistema propuesto.

3. Metodología

Componente 1 — Captura de tráfico real. Montaremos una red controlada propia (máquinas virtuales en red aislada y, complementariamente, una topología en GNS3) para garantizar que no se toca infraestructura de terceros. Un generador producirá tráfico representativo: sesiones TCP completas (cliente/servidor con sockets), datagramas UDP y los escaneos de puertos a estudiar (con nmap y con un escáner propio en scapy). La captura se hará con scapy y se guardará como archivos .pcap reproducibles.

Componente 2 — Implementación a bajo nivel. El núcleo propio es un parser que decodifica las cabeceras TCP y UDP a partir de los bytes crudos (módulo struct de Python), sin usar el disector automático de scapy para interpretar los campos. Sobre el parser, un reconstructor de estados seguirá el acuerdo de tres vías, la evolución de la ventana, las retransmisiones y el cierre de cada flujo. La correctitud se demuestra comparando, campo a campo, la salida del parser propio contra tshark/pyshark sobre el mismo .pcap.

Componente 3 — Análisis de la anomalía. El detector reconstruye, por dirección de origen, los intentos de conexión y aplica heurísticas de escaneo (muchos puertos contactados en una ventana temporal corta, alta proporción de RST/SYN-ACK, conexiones semiabiertas). Reproduciremos varios tipos de escaneo en nuestra red, con verdad de referencia conocida (sabemos cuándo y cómo se lanzó cada escaneo), para medir la efectividad de la detección con métricas honestas.

4. Plan de trabajo interno

El equipo se distribuye en tres roles complementarios; el informe, la presentación y los commits se reparten entre los tres integrantes. *(Asignar cada nombre al rol correspondiente antes de la entrega.)*

Rol	Responsabilidades principales
Infraestructura y captura	Topología de red controlada, generador de tráfico, captura y curado de archivos .pcap, gestión del repositorio Git.
Implementación de bajo nivel	Parser de cabeceras TCP/UDP sobre bytes crudos, reconstructor de estados, validación campo a campo contra Wireshark/tshark.
Análisis y detección	Detector de escaneo de puertos, definición y cálculo de métricas, gráficos de resultados y elaboración del póster.

Cronograma del equipo

Fase	Tarea	Fechas
1. Base	Estudio de RFC, montaje de red, generador de tráfico, repositorio inicial	13–15 jun
2. Núcleo	Parser de bajo nivel TCP/UDP, reconstructor de estados, validación vs tshark	16–18 jun
3. Detección	Detector de escaneo, métricas, .pcap definitivos, README y guion de pruebas	19–21 jun
4. Cierre	Póster A1, ensayo de la demostración en vivo	22 jun
E3	Presentación oral y demostración	23 jun

5. Métricas de éxito

- **Correctitud del parser:** porcentaje de campos TCP/UDP que coinciden con tshark sobre el mismo .pcap (objetivo $\geq 99\%$ en los campos clave).
- **Calidad de la detección:** precisión, exhaustividad (recall) y F1; tasa de falsos positivos por hora de tráfico normal.
- **Latencia de detección:** tiempo entre el inicio de un escaneo y la alerta correspondiente.
- **Cobertura:** tipos de escaneo detectados frente a los probados (SYN, connect, FIN/NULL/Xmas, UDP).
- **Reproducibilidad:** el docente ejecuta el guion de pruebas y obtiene resultados equivalentes a los reportados.

6. Riesgos previstos y mitigación

Riesgo	Impacto	Mitigación
Falsos positivos altos en la detección	Medio	Afinar umbrales y ventana temporal; lista de hosts conocidos; contrastar con conjuntos CIC-IDS.
Tráfico capturado poco representativo	Medio	Generar escenarios variados de TCP/UDP; documentar la metodología de captura.
Permisos de captura (root / Npcap)	Bajo	Usar VM con privilegios; documentar el setup en el README.
Desincronización del equipo en Git	Medio	Commits frecuentes, ramas por componente, integración temprana.
Tiempo limitado al 23 de junio	Alto	Priorizar un MVP (handshake + escaneo SYN); UDP scan y SYN flood como extensiones.

7. Asistencia de IA

Se utilizó un asistente de IA generativa como apoyo en la redacción y estructuración de esta propuesta y del diagrama de arquitectura. Todo el diseño técnico, la implementación del código y los

resultados experimentales serán desarrollados y verificados por el equipo. *(Detallar aquí qué se solicitó y cómo se incorporó, conforme al enunciado.)*

8. Referencias

- RFC 9293 — Transmission Control Protocol (TCP). IETF, 2022.
- RFC 768 — User Datagram Protocol. IETF, 1980.
- RFC 4987 — TCP SYN Flooding Attacks and Common Mitigations. IETF, 2007.
- RFC 5681 — TCP Congestion Control. IETF, 2009.
- Kurose, J. y Ross, K. Computer Networking: A Top-Down Approach (8.^a ed.).
- Wireshark / tshark (wireshark.org) y scapy (scapy.net).